

Now I have everything current. Let me generate the full outline.

Course Outline: Skill Creation

Title: Skill Creation **Learnpack:** + Creación de Skills **Prerequisite:** Skills: ¿Cómo puede tu agente aprender habilidades casi como Neo en Matrix? (same day, previous learnpack) **Target Audience:** Beginner AI engineers who already understand what agent skills are and want to learn how to create their own **Total Lessons:** 11 **Format:** Conceptual/Instructional

Course Description

This course teaches students the complete process of creating agent skills — from planning and writing the SKILL.md file to registering, testing, and iterating on the result. Students already know what skills are and where to find existing ones from the previous learnpack; now they learn to build their own.

Agent Skills follow an open standard (agentskills.io) adopted by over 30 platforms including Claude Code, Cursor, GitHub Copilot, and ChatGPT Codex. This means a skill written once works across multiple agents. The course teaches this universal SKILL.md format and shows students where to place skills in each of the major tools they use.

The course uses a single running example — building a README generator skill — that progresses across the building section, so students see the full creation process unfold naturally before building their own skill at the end.

Course Philosophy

This course emphasizes:

- **Spec-first thinking:** Plan what the skill does before writing it
- **Path of least resistance:** Start with the simplest skill that works, iterate from there
- **Attention to detail:** Small wording differences in a SKILL.md dramatically change agent behavior
- **Autonomous problem-solving:** Students learn to evaluate and improve their own skills through testing

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - What Is a Skill?	2
02 - Building a Skill	4
03 - Test and Build	2

Section	Lessons
04 - Assessment	1
05 - Conclusion	1
Total	11

Section 00: Welcome

00.0 Welcome

Learning Objectives:

- Understand what this course covers and what you'll be able to do by the end
- Connect this course to what you already know about skills from the previous learnpack

Content Outline:

- Quick recap: you already know what skills are, how agents use them, and where to find existing ones (skills.sh, community marketplaces)
- This course is about creating your own — the full process from blank file to working skill
- What we'll cover: the SKILL.md file format, planning, writing, registering across agents (Claude Code, Cursor, Copilot, Codex), and testing
- The Agent Skills open standard (agentskills.io): skills you create are portable across 30+ platforms — write once, use everywhere
- By the end: you'll have built and tested a complete skill yourself

Transitions: Leads into Section 01, where we look at what's actually inside a skill file before building one.

Key Principles:

- A skill is a focused instruction set that teaches your agent a specific capability
- Creating skills is a design activity — writing clear instructions matters more than technical complexity

Examples:

- A skill that generates README files following a specific template
 - A skill that scaffolds API endpoints with consistent structure
-

Section 01: What Is a Skill?

01.0 Anatomy of a Skill File

Learning Objectives:

- Identify the components that make up a skill: SKILL.md, optional directories (scripts/, references/, assets/)
- Explain how agents discover and load skills using progressive disclosure

Content Outline:

- A skill is a directory containing a SKILL.md file — that's the only required piece
- The SKILL.md has two parts: YAML frontmatter (between `---` markers) and a markdown body with instructions
- Required frontmatter fields: `name` (the identifier, becomes the slash command) and `description` (tells the agent when to activate it)
- Optional frontmatter: `license`, `metadata` (author, version), `compatibility`
- The markdown body: the actual instructions the agent follows when the skill is activated
- Optional supporting directories:
 - `scripts/` — executable code the agent can run (Python, Bash, JS)
 - `references/` — additional documentation the agent reads on demand
 - `assets/` — templates, config files, examples
- How progressive disclosure works: only name and description load at startup (~30-50 tokens per skill). The full SKILL.md loads only when activated. Reference files load only when needed during execution. This means you can have hundreds of skills without wasting context
- Walk through a real, simple skill directory to see each part

Transitions: Now that you know what's inside a skill, the next lesson covers when it makes sense to create one — and when it doesn't.

Key Principles:

- The description is the "when" — the SKILL.md body is the "how"
- Progressive disclosure keeps your context window clean — skills only consume tokens when they're actually being used
- Keep SKILL.md under 500 lines / 5,000 tokens. If it's longer, split content into reference files

Examples:

- A complete skill directory: `readme-generator/SKILL.md`, `readme-generator/assets/template.md`
- The frontmatter for a simple skill:

```
---
name: readme-generator
description: Generates a project README following team conventions. Use when
asked to create, write, or generate a README for a project.
---
```

01.1 Why and When to Create One 📖

Learning Objectives:

- Identify situations where creating a custom skill is the right choice
- Recognize when a skill is overkill and simpler alternatives suffice
- Distinguish between skills and rules (connecting to Day 17 concepts)

Content Outline:

- When to create a skill: repeatable tasks you do more than twice, tasks with a clear input→output pattern, tasks where you want consistent quality every time
- When NOT to create a skill: one-off tasks, tasks that are too broad to scope clearly, tasks better handled by a rule or a direct prompt
- Skills vs. rules: rules are always-on behavioral constraints (code style, naming conventions). Skills are on-demand procedural knowledge that activates only when relevant. Rules shape how the agent works generally; skills teach it how to do a specific job. This connects directly to what you learned in Day 17 about rules and context engineering — skills are the next step
- The single-responsibility principle: one skill = one job. If you can't describe what it does in one sentence, it's too broad
- **Anti-pattern — God Skill:** a monolithic skill that tries to handle everything (create components, write tests, update docs, deploy). It produces unpredictable output because it has too many responsibilities. Fix: split into smaller, focused skills
- **Anti-pattern — Scope creep:** starting with "generate a README" and gradually adding "also update the changelog, check for broken links, and validate the CI config." Each addition dilutes the skill's reliability

Transitions: You know what a skill is and when to build one. Section 02 walks through the creation process step by step, using a single running example.

Key Principles:

- One skill, one responsibility — if it does two things, split it into two skills
- The test: "Would I explain this process to a colleague the same way every time?" If yes, it's a skill candidate
- Skills are an evolution of rules — more focused, more procedural, activated on demand

Examples:

- Good skill candidate: "Generate a project README following our team's template and conventions"
- Bad skill candidate: "Help me with my project" — too vague, no clear input/output
- Borderline: "Add error handling to functions" — this is probably a rule (always do this) not a skill (do this specific multi-step process)

Section 02: Building a Skill

This section uses a single running example — building a README generator skill — that progresses across all four lessons.

02.0 Planning Inputs, Outputs & Triggers

Learning Objectives:

- Define the inputs a skill expects from the user or context
- Define the outputs the skill must produce
- Identify the trigger phrases or conditions that should activate the skill

Content Outline:

- Before writing anything: answer three questions — What does the user provide? What does the skill produce? What request should activate it?
- Inputs: what information must be present for the skill to work. For our README generator: the project files in the current directory. Define what's required vs. optional — required: project files exist; optional: an existing README to update
- Outputs: what the skill delivers and in what format. For our README generator: a README.md file following a specific template structure. Be specific: "a markdown file" is vague; "a README.md with sections for Description, Setup, Usage, and Contributing" is actionable
- Triggers: the phrases or situations that should activate this skill. For our README generator: "generate readme", "create project documentation", "write a readme". Also think about what should NOT trigger it
- The Spec → Plan → Execute pattern: specify what the skill does before writing instructions
- Validate your plan: can you describe the input→output in one sentence? "Given a project directory, produce a README.md with Description, Setup, Usage, and Contributing sections."
- **Anti-pattern — Implicit inputs:** the skill depends on context that isn't declared, like assuming a specific file structure exists without checking. It works sometimes and fails mysteriously other times. Fix: make every required input explicit in the SKILL.md
- **Anti-pattern — Ambiguous outputs:** the skill produces open-ended responses when structured output is needed. "Write some documentation" vs. "Generate a README.md with these specific sections." Fix: specify exact output format
- **Anti-pattern — Under-specification:** the plan is too vague — "make a readme from the project" leaves the agent guessing about structure, length, and content. Fix: define concrete deliverables before writing

Transitions: With inputs, outputs, and triggers planned, we're ready to write the actual SKILL.md file.

Key Principles:

- Inputs and outputs must be explicit — never depend on implicit context the agent might not have
- If you can't describe the input→output in one sentence, the skill scope is too broad

Examples:

- Our running example plan: Input = project files in current directory. Output = README.md following template. Triggers = "generate readme", "create project docs", "write readme"
- A second example for contrast: an API scaffolder — Input = entity name and fields. Output = route file and controller. Triggers = "scaffold endpoint for [entity]"

02.1 Writing the SKILL.md

Learning Objectives:

- Write a SKILL.md file that an agent can follow reliably
- Apply conventions for writing instructions targeted at an AI audience: imperative tone, unambiguous language, structured steps

Content Outline:

- The SKILL.md body is the instruction manual the agent reads when the skill activates — everything it needs must be here

- Writing for an AI audience: use imperative verbs ("Read", "Create", "Validate"), avoid ambiguity ("appropriate" → specify what counts as appropriate), be explicit about order of operations
- Structure conventions: start with a clear purpose statement, list step-by-step instructions, specify output format, include constraints and edge cases
- Including examples inside the SKILL.md: show the agent what good output looks like. A concrete example is worth more than paragraphs of instruction. For our README generator, include a sample output showing the expected structure
- The "minimum context, maximum clarity" principle: include only what the agent needs, but include ALL of what it needs. Remember the 500-line / 5,000-token guideline — if your instructions are getting long, move reference material into a `references/` directory
- Referencing supporting files: "Read the template at `assets/template.md` and use it as the base structure." The agent loads these on demand, so they don't waste context until needed
- Separate "thinking" from "doing": if the agent needs to analyze before acting, make that an explicit step. "Step 1: Read the project's package.json and identify the tech stack. Step 2: Based on the tech stack, select the appropriate template section..."
- **Anti-pattern — Overprompting:** stuffing the SKILL.md with irrelevant instructions that dilute the important ones. A README generator skill that includes three paragraphs about coding philosophy wastes the agent's attention. Fix: remove anything the agent doesn't need for THIS specific task
- **Anti-pattern — Mixed responsibilities:** one skill that decides what to document, writes the README, validates links, and publishes to a wiki. Fix: separate concerns — one skill generates, another validates

Transitions: The SKILL.md body tells the agent what to do. Next, we write the description that tells the agent when to do it.

Key Principles:

- Write instructions as if the reader has no memory of previous conversations — every activation starts fresh
- Examples inside the SKILL.md are the single most effective way to improve output quality
- If it's getting long, split into SKILL.md (core instructions) + references/ (detailed docs)

Examples:

- Our running example — the README generator SKILL.md body:

Generate a README.md for the current project.

1. Read the project's root directory. Identify the tech stack from `package.json`, `requirements.txt`, or equivalent config files.
2. Read the template at `assets/template.md`.
3. Fill in each section of the template:
 - Description: one paragraph summarizing what the project does
 - Setup: installation steps based on the detected package manager
 - Usage: basic usage example based on the entry point
 - Contributing: standard contributing guidelines
4. Write the result to `README.md` in the project root.

If a `README.md` already exists, read it first and preserve any custom sections not in the template.

- A poorly-written version of the same: "Look at the project and write a good README." — the agent doesn't know what sections, what format, or what "good" means
-

02.2 Writing the Description

Learning Objectives:

- Write a skill description that triggers accurately on relevant requests and stays silent on irrelevant ones
- Use positive and negative trigger language to control activation precision

Content Outline:

- The description field in the YAML frontmatter is the gatekeeper: the agent reads all skill descriptions at startup and uses them to decide which skill to activate for a given request
- The description should answer: "What does this skill do, and when should the agent use it?"
- Positive triggers: phrases, keywords, and scenarios where this skill SHOULD activate. Be specific and enumerate variations. For our README generator: "Use when asked to create, write, or generate a README, project documentation, or project description file"
- Negative triggers: scenarios where this skill should NOT activate. This prevents false positives. For our README generator: "Do NOT use for API documentation, changelogs, or general writing tasks"
- The mental model test: run through 5-10 realistic user messages mentally and check which ones would trigger it. "Write me a readme" → yes. "Document this API endpoint" → no. "Create project docs" → yes. "Write a blog post about the project" → no
- Specificity vs. over-triggering: a description that's too broad fires on everything; too narrow never fires
- The 200-character recommendation: descriptions should be concise. The agent processes all skill descriptions at startup — shorter means more efficient matching
- **Anti-pattern — Missing negative triggers:** the description says "Use when working with documentation" — this fires on API docs, changelogs, inline comments, and anything documentation-adjacent. Fix: add explicit negative boundaries
- **Anti-pattern — Over-triggering:** the description is so broad that the skill hijacks unrelated requests. A README generator that fires every time someone mentions "project" or "file." Fix: be specific about the action, not just the domain
- **Anti-pattern — Vague descriptions:** "A useful skill for projects" — the agent has no idea when to activate this. Fix: describe the specific action and trigger conditions

Transitions: You've written the instructions and the trigger. Now let's put the skill in the right place so your agent can find it.

Key Principles:

- Negative triggers are as important as positive ones — they prevent your skill from hijacking unrelated requests
- Test your description mentally: simulate 10 user messages and check which ones would fire it
- Keep it under 200 characters — concise descriptions match more reliably

Examples:

- Our running example — the complete frontmatter:

```

---
name: readme-generator
description: Generates a project README from the codebase. Use when asked to
create, write, or generate a README. Do NOT use for API docs, changelogs, or
blog posts.
---
```

- An over-triggering description: "Use when the user mentions documentation" — fires on everything
- An under-triggering description: "Use only when the user types 'please generate a README.md file for my project'" — too literal, misses natural variations

02.3 Where Your Skills Live 📖

Learning Objectives:

- Register a skill in each of the four major agents: Claude Code, Cursor, GitHub Copilot, and ChatGPT Codex
- Distinguish between project-scoped and personal-scoped skills
- Verify that the agent can discover your skill after registration

Content Outline:

- All four major agents use the same SKILL.md format (the Agent Skills open standard). The only difference is where you put the directory
- Directory conventions per agent:

Claude Code:

- Project skills: `.claude/skills/<skill-name>/SKILL.md`
- Personal skills: `~/.claude/skills/<skill-name>/SKILL.md`
- Invoke with `/skill-name` or let Claude activate automatically from the description
- Hot-reloading: saving the file updates the skill immediately — no restart needed

Cursor:

- Project skills: `.cursor/skills/<skill-name>/SKILL.md`
- No frontmatter required (simpler than Cursor rules)
- Cursor also has `.cursor/rules/*.mdc` files — rules are always-on context, skills are on-demand. Don't duplicate the same instruction in both

GitHub Copilot:

- Project skills: `.github/skills/<skill-name>/SKILL.md`
- Copilot also reads `.github/copilot-instructions.md` (repo-wide rules) and `AGENTS.md` (cross-agent instructions). Skills are the on-demand layer on top of these
- Available in VS Code, JetBrains, and the Copilot cloud agent

ChatGPT Codex:

- Project skills: `.agents/skills/<skill-name>/SKILL.md`
- Codex also reads `AGENTS.md` for repo-wide instructions
- Invoke with `/skills` or `$skill-name`, or let Codex activate implicitly
- Project vs. personal scope: project skills live in the repo and apply to everyone on the team. Personal skills live in your home directory and apply only to you. Use project skills for team conventions, personal skills for your individual workflows
- Portability: because the SKILL.md format is identical across agents, you can symlink the same skill folder into multiple agent directories if you use more than one tool
- Verification: after placing the skill, verify the agent can see it. In Claude Code: run `/skills`. In Cursor: check the skills panel. In Copilot: look for the skill in the Chat Customizations editor. In Codex: run `/skills` or type `$`
- For our running example: place the `readme-generator/` folder in the appropriate directory for your agent, then verify it appears in the skill list

Transitions: Your skill is built and registered. Section 03 covers how to verify it actually works and how to iterate on it.

Key Principles:

- Same format, different directories — the SKILL.md is identical everywhere, only the path changes
- Project skills travel with the repo (team-shared). Personal skills are local to you
- Always verify after registration — check the agent's skill list before testing

Examples:

- Our running example — the complete file structure for Claude Code:

```
.claude/skills/readme-generator/  
├── SKILL.md  
└── assets/  
    └── template.md
```

- The same skill registered for Codex:

```
.agents/skills/readme-generator/  
├── SKILL.md  
└── assets/  
    └── template.md
```

Section 03: Test and Build

03.0 Testing Your Skill

Learning Objectives:

- Test a skill by observing activation, execution, and output quality
- Diagnose common failures by tracing them back to the description, the SKILL.md, or missing input
- Iterate on a skill based on test results

Content Outline:

- The testing loop: trigger the skill → observe if it activated → evaluate the output → adjust → repeat.
Most skills need 2-3 rounds of iteration
- **Testing activation:** Does the skill fire when it should? Try the trigger phrases you planned in 02.0. Also try edge-case prompts that are close but shouldn't trigger — "write me a changelog" should NOT activate the README generator
- **Testing execution:** Does the agent follow the instructions in order? Does it use the template file? Does it produce the expected sections? Compare the output against your expected result
- **Testing edge cases:** What happens with missing input (empty project, no package.json)? What about an existing README — does it preserve custom content as instructed?
- **Diagnosing failures — a simple flowchart:**
 - Skill didn't activate? → Problem is in the **description**. Check trigger wording, check negative triggers aren't too broad
 - Skill activated on the wrong request? → Problem is in the **description**. Negative triggers are missing or too narrow
 - Skill activated but output is wrong? → Problem is in the **SKILL.md body**. Instructions are ambiguous, missing steps, or missing examples
 - Skill activated but crashed or errored? → Problem is **missing input or broken references**. A referenced file doesn't exist, or the project structure doesn't match assumptions
- Tool-specific testing notes:
 - Claude Code: hot-reloading means you can edit SKILL.md and immediately re-test without restarting
 - Cursor: if an update doesn't appear, try reloading the window
 - Codex: detects changes automatically, but may need a restart if updates don't appear
- The quality checklist as a testing aid — run through these questions for your skill:
 - Does it have a single, clear objective?
 - Are inputs defined and validated?
 - Is the output structured and verifiable?
 - Does the description include negative triggers?
 - Did I include examples in the SKILL.md?
 - Have I tested with at least one edge case?
- **Anti-pattern — Untested edge cases:** the skill works perfectly on the happy path but breaks on empty directories, projects without config files, or unexpected file structures. Fix: test at least three scenarios — happy path, missing input, and edge case

Transitions: You now know how to build, register, test, and fix skills. Next, you'll build your own skill from scratch using everything you've learned.

Key Principles:

- A skill is never done after the first draft — expect to iterate 2-3 times

- When something goes wrong, trace it: description problem → SKILL.md problem → input problem
- Test with edge cases early — they reveal ambiguity faster than happy-path tests

Examples:

- Testing the README generator: try it on a Node.js project, a Python project, and an empty directory
 - Discovering an issue: the skill ignores an existing README. Trace: the output is wrong → SKILL.md problem → add an instruction to read existing README first and preserve custom sections
-

03.1 Build Your Own Skill

Content Outline:

- Guided end-to-end exercise: design, write, and test a complete skill from scratch
- Step 1: Choose a repeatable task you do with your agent (or pick from suggested options: component scaffolder, commit message writer, code review checklist, test generator)
- Step 2: Plan — answer the three questions: What input? What output? What triggers?
- Step 3: Write the SKILL.md — follow the conventions from lesson 02.1, include at least one example in the instructions
- Step 4: Write the description — include positive and negative triggers, keep it under 200 characters
- Step 5: Set up the directory in your agent of choice, add any supporting assets
- Step 6: Test — run at least 3 test cases: happy path, edge case, and a request that should NOT trigger the skill
- Step 7: Iterate — based on test results, adjust the description or instructions. Expect 2-3 rounds
- Self-review checklist:
 - Does my skill have a single, clear objective?
 - Are inputs defined and validated in the instructions?
 - Is the output structured and verifiable?
 - Does the description include negative triggers?
 - Did I include examples in the SKILL.md?
 - Is the SKILL.md under 500 lines?
 - Does it activate correctly and stay silent when it should?
 - Have I tested with at least one edge case?

Note: This is a guided practice exercise — no theory, no new concepts. The student applies everything from the course.

Section 04: Assessment

04.0 Skill Design Quiz 🧐

Learning Objectives:

- Demonstrate understanding of skill anatomy, creation process, quality criteria, and common pitfalls
- Apply judgment to realistic skill design scenarios

Question Format: Scenario-based

Questions:

1. You have a skill that generates API endpoint scaffolding. It works perfectly for REST endpoints but also fires when a user asks for help writing API documentation. What is the most likely cause and how would you fix it?
2. A colleague shows you their SKILL.md. It starts with valid frontmatter but the body just says: "Create a good test file for the user's code." What are two specific problems with this instruction, and how would you rewrite it?
3. You're creating a skill to generate database migration files. Before writing the SKILL.md, what are the three planning questions you should answer? Provide example answers for this specific skill.
4. A skill's SKILL.md is 800 lines long and covers creating components, writing tests, updating documentation, and deploying to staging. Which anti-pattern is this, and what is the recommended fix?
5. You've written a skill and placed it in `.claude/skills/my-skill/SKILL.md`, but when you test it, the agent never activates it — even when you use the exact trigger phrases. Name two possible causes.
6. You want to use the same skill in both Claude Code and Codex. What do you need to change in the SKILL.md file to make it work in both? Where does the skill directory need to be placed for each agent?

Evaluation Criteria:

- Correct identification of problems and anti-patterns
- Practical, specific fixes (not vague advice)
- Demonstrates understanding of the description vs. SKILL.md body distinction
- Shows awareness of cross-agent directory conventions

Score Interpretation:

- 6/6: Ready to create production-quality skills independently
- 4-5/6: Solid understanding, review the areas you missed
- Below 4/6: Revisit Sections 02 and 03 before proceeding

Section 05: Conclusion

05.0 Your Agent Leveled Up

Content Outline:

- Course recap: you went from understanding skills to building, testing, and iterating on your own
- What you can now do: plan a skill, write a SKILL.md with proper frontmatter and instructions, write an effective description with positive and negative triggers, register it in Claude Code, Cursor, Copilot, or Codex, test it, and diagnose problems
- The open standard advantage: every skill you create works across 30+ platforms. As new agents adopt the Agent Skills standard, your skills travel with you
- Connection to the bigger picture: skills are one part of your agent configuration toolkit alongside rules (Day 17), context, and specifications. Knowing when and how to use each one makes you a more effective AI engineer

- Next steps: keep building skills as you encounter repeatable tasks, share useful skills with your team by committing them to the repo, explore community skill directories for inspiration
- Resources: agentskills.io (open standard specification), skills.sh (skill directory), community marketplaces
- Encouragement: your first skill won't be perfect — the testing loop is where the quality comes from. Every skill you build makes the next one easier

Note: This is conclusion only — no theory, exercises, or assessment.
